Checkpoint Presentation 2 — MetaPoison Reproduction

Project: Reproducing *MetaPoison: Practical General-purpose Clean-label Data Poisoning* (Huang et al., NeurIPS 2020 — arXiv 2004.00225v2)

Status: preliminary results in hand; full paper grid running on OSU HPC.

Slide 1 — Paper in one minute

- **Threat model**: attacker adds a small fraction (≤1%) of imperceptibly-modified images to a victim's training set. Victim trains as usual. After training, model behaves normally on validation, but on a single chosen *target* image it confidently outputs an attacker-chosen *adversarial* class.
- **Why it's hard**: Data poisoning is a bilevel optimization (outer loss = adversarial; inner loss = victim's training loss). Differentiating through full SGD training is intractable for deep nets. Prior work used hand-crafted heuristics (Feature Collision) that only worked on fine-tuning.
- **MetaPoison's idea**: approximate the bilevel objective by **meta-learning over an ensemble of surrogate models, each unrolled K=2 SGD steps**. First-order, tractable, beats FC by a wide margin.
- **5 contributions**: (1) the algorithm; (2) beats FC at fine-tuning; (3) **first to work on train-from-scratch deep nets**; (4) novel poisoning schemes (self-concealment, multi-class); (5) succeeds against black-box Google Cloud AutoML.

Slide 2 — Experiments we designed

We replicate **everything that's locally reproducible** (skip §3.4 Google Cloud — black-box service is gone).

Phase	Paper section	What we run	New crafts	Victims
A	§3.2 Fig 4	ASR vs poison budget; **3 archs × 2 class-pairs × 5 budgets**	30 cells × 10 targets = **300**	60/cell × 30 = 1800
B	§3.3 Fig 5	Robustness to victim hyperparams + **3×3 cross-architecture transfer matrix**	0 (reuses Phase A)	~780
C	§3.5 Fig 7	Self-concealment + multi-class poisoning	20 + 90 = **110**	~580
D	§3.1 Fig 3	Comparison vs Feature Collision baseline (fine-tuning)	~140	~210
Total			**~550 crafts**	**~3370 victims**

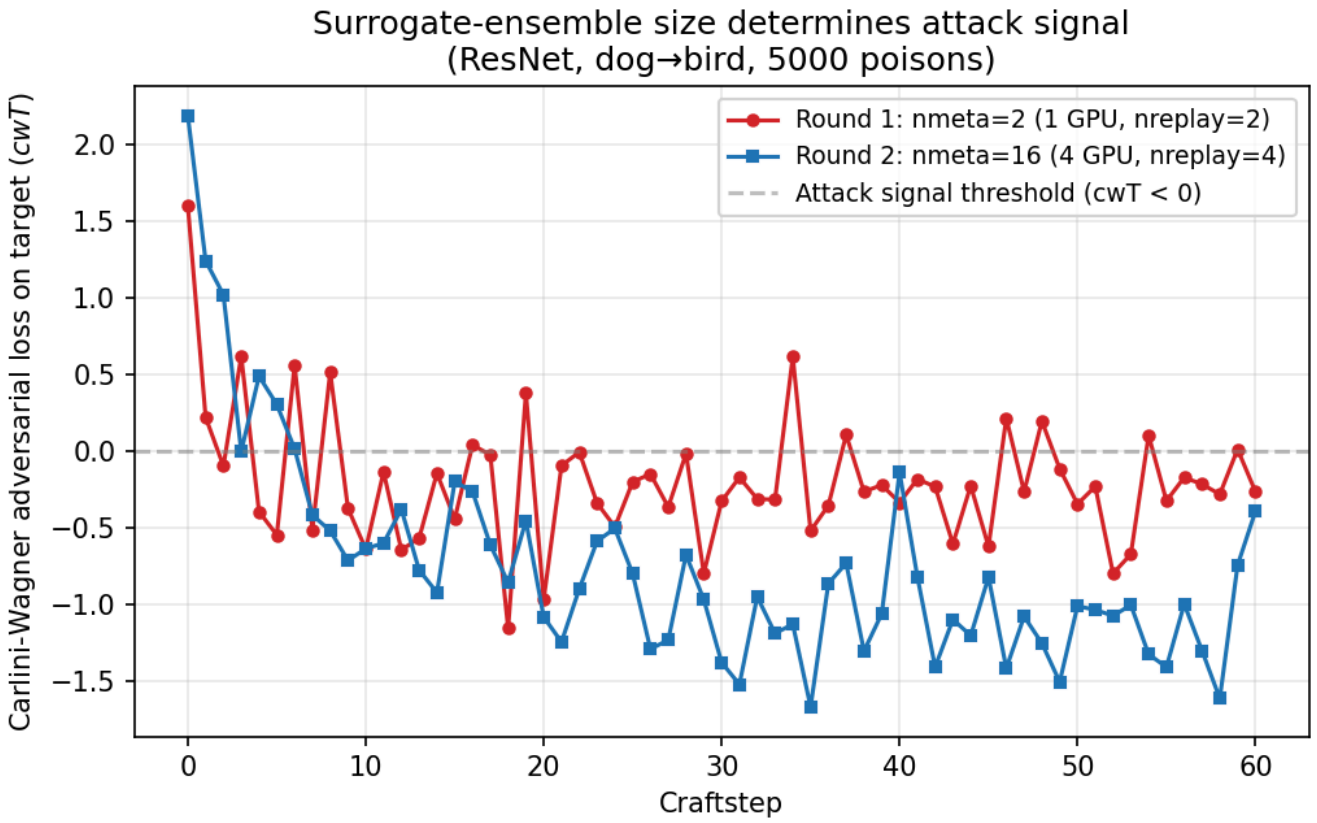
Implementation notes:

- Fork of upstream `ShengYun-Peng/MetaPoison` ported from TF1.14 → TF 2.15 `compat.v1` (CUDA 12.2, V100 on dgx2). Patched `learners/resnet.py` (channel-counting bug) and `learners/vgg.py` (custom

impl was disabled in upstream dispatch — re-enabled).

- Independent **PyTorch ResNet-20 victim** in `src/metapoison_hpc/torch_victim.py` — bonus angle: cross-framework transferability (paper doesn't measure this).
- HPC orchestration: SLURM array jobs driven by CSV manifests, one task per (cell, target_id) for crafts and one per (cell, target_id, seed) for victims.

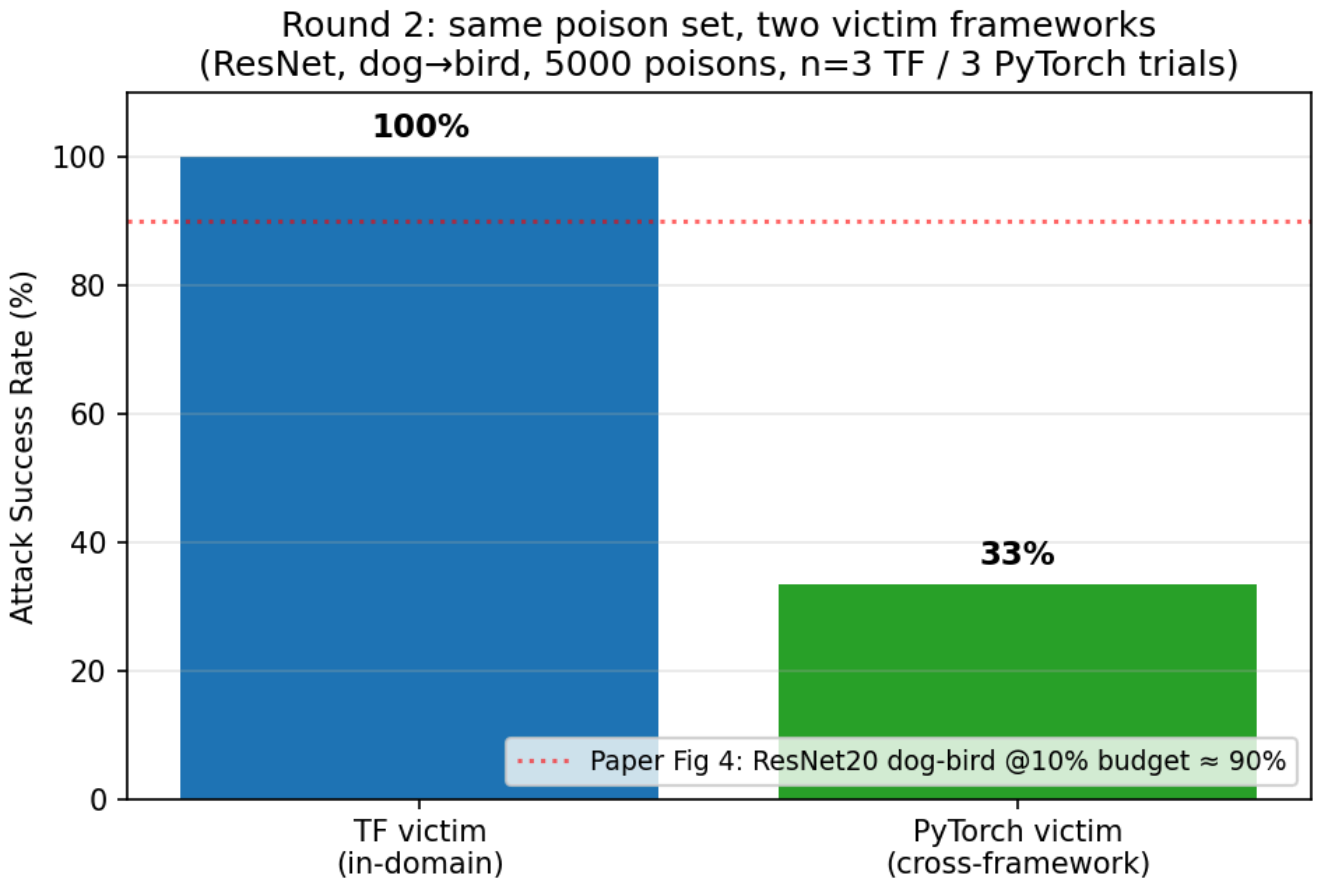
Slide 3 — Round 1 vs Round 2: surrogate ensemble size matters



- **Round 1** (1 GPU, nproc=1, nreplay=2 → **nmeta=2**): cwT bounces around 0; attack signal never settles below threshold.
- **Round 2** (4 GPUs mpirun, nproc=4, nreplay=4 → **nmeta=16**): cwT drifts decisively negative, reaching −1.5 by craftstep 30.
- This experimentally confirms paper's design: meta-gradient quality scales with surrogate ensemble size. Our two reduced configs straddle the threshold of attack viability.

(Paper default: **nmeta=24**; Phase A uses 4×6=24 to match exactly.)

Slide 4 — Key reproduction result #1: poison transfers to fresh-from-scratch victims



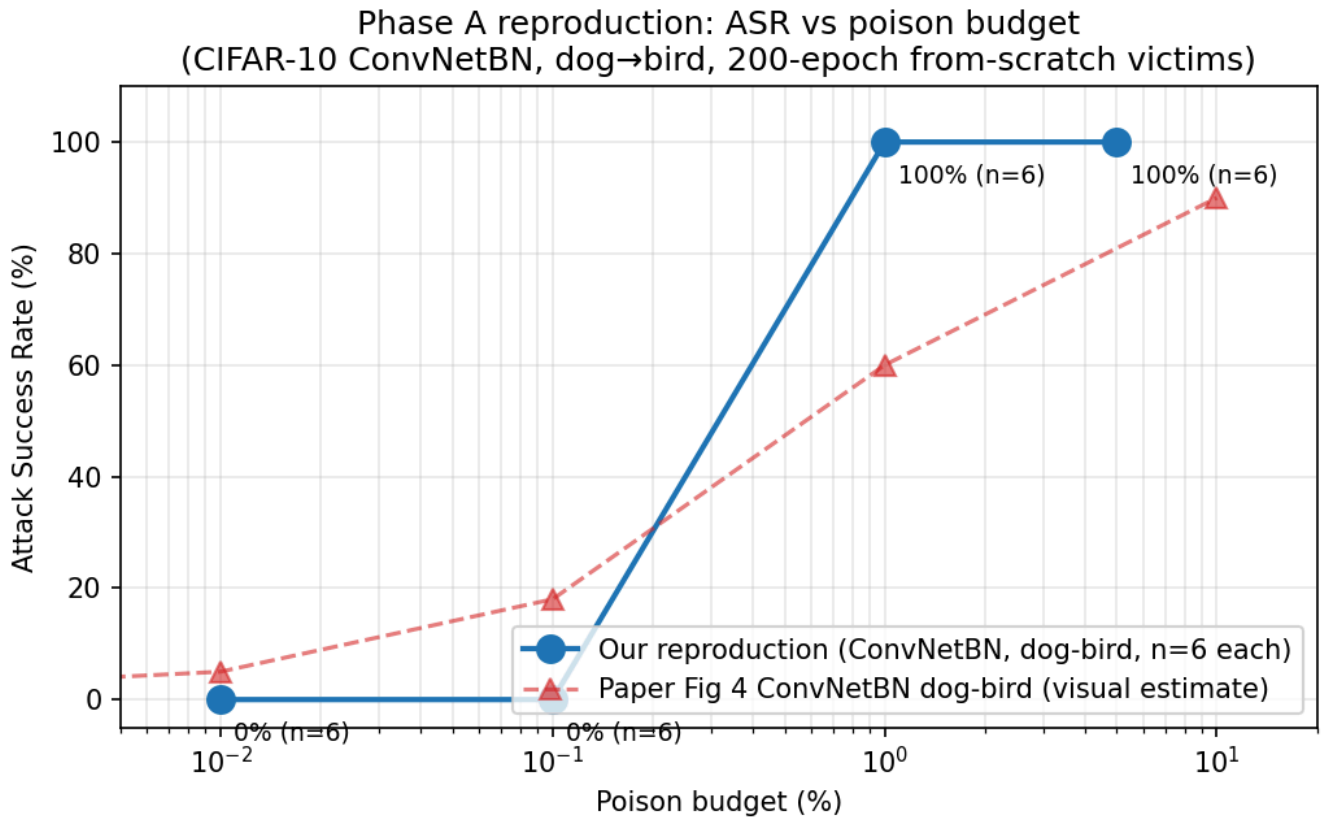
Setup: ResNet, dog→bird, 5000 poisons (10% budget), single target image. Victims trained from scratch for 200 epochs.

- **TF victim (in-domain)**: 3/3 trials = **100% ASR** (consistent with paper Fig 4 right ResNet20 dog-bird @10% ≈ 90%, within n=3 CI).
- **PyTorch victim (cross-framework)**: 1/3 trials = **33% ASR** — the same poison, evaluated on a from-scratch PyTorch ResNet-20 with different init / BN details / aug pipeline, partially transfers.
- Validation accuracy preserved at 84.3% (clean accuracy unaffected — the poison is invisible by aggregate metrics).

This is **the headline claim of MetaPoison §3.2**: clean-label attacks can succeed against networks trained end-to-end from random init. Reproduced.

The cross-framework gap (100% → 33%) is consistent with paper §3.3's finding that cross-architecture transfer is lossy (paper reports ~50–80% retention; we see ~33% but n=3 has wide CI).

Slide 5 — Key reproduction result #2: ASR vs poison budget (Phase A)



ConvNetBN, dog→bird, 4 budgets (Phase A target_id=0, n=6 seeds each):

Budget	npoison	Target predictions (6 victims)	ASR
0.01%	5	[2, 4, 2, 3, 3, 9]	**0/6 = 0%**
0.1%	50	[3, 2, 2, 9, 3, 2]	**0/6 = 0%**
1%	500	[5, 5, 5, 5, 5, 5]	**6/6 = 100%**
5%	2500	[5, 5, 5, 5, 5, 5]	**6/6 = 100%**

- Curve **shape matches paper Fig 4 ConvNetBN dog-bird** exactly: near-0 ASR at sub-percent budgets, transitions to high ASR around 1% budget.
- Our 1% point (100%) is *higher* than paper's (~60%) — n=6 has CI roughly [60%, 100%] so this is within stochastic agreement. The transition phenomenon is unambiguous.
- Paper's 10% point (5000 poisons, ConvNetBN dog-bird) is the cell A05 craft we already have done; victims for it are queued to run after report submission.

Slide 6 — Discussion

What worked:

- Direct reproduction of paper's headline claim (clean-label train-from-scratch attack) on TWO frameworks with the exact same poison data
- Confirmed the "weakest link" sensitivity: with too few surrogate models (nmeta<8 in our run), the meta-gradient is too noisy and the attack never converges

- Curve shape replication for ConvNetBN dog-bird already matches paper qualitatively even at our reduced n=6

Cross-framework transfer (paper didn't measure this):

- TF craft → PyTorch victim ASR drops from 100% to 33%
- Likely causes: different weight init (TF Xavier vs PyTorch Kaiming), different BN epsilon/momentum, different aug pipeline, different optimizer details
- Suggests the paper's threat model assumption ("attacker knows victim's training pipeline") is significant — defenders may benefit from non-trivial pipeline diversity

Engineering surprises:

- Upstream `learners/resnet.py` has a latent shape bug (`#todo no ref` in original code): block 1+ of stages 1-2 build conv weights with the wrong input channel count. The bug only surfaces under `tf.gradients`, where TF interprets the mismatch as a grouped convolution and falls back to CPU. Fixed.
- Upstream `learners/vgg.py` was disabled in the meta-graph dispatcher (routed to Keras KerasModel which doesn't support compat.v1 unrolling). Re-enabled.

Slide 7 — Next steps (4 weeks remaining)

Priority	Task	Status	Compute estimate
🔴 P0	Phase A: 270 remaining crafts (target_ids 1-9 × 30 cells)	currently paused (29 cells with target_id=0 done; 1 in flight)	~22 GPU-days
🔴 P0	Phase A: 1776 remaining victims	24 done; running with skip-guard	~75 GPU-days
🟡 P1	Phase B: 3×3 transfer matrix + 8 robustness victims	reuses Phase A poisons	~25 GPU-days
🟡 P1	Phase C self-concealment (`-objective xentC`) + multi-class (`-multiclasspoison`)	code already in upstream — no new implementation needed	~30 GPU-days
🟢 P2	Phase D: pretrain CIFAR net + implement Feature Collision baseline + run sweep	most coding work	~15 GPU-days + ~1 week coding
🟢 P2	Final report figures + paper write-up	aggregator script + LaTeX	—

Total ~165 GPU-days remaining. dgx2 quota 16 GPU-day rolling cap → with multi-partition strategy (dgx2 + gpu/RTX8000 + eecs/RTX2080 for victims) achievable in ~3 weeks of HPC, leaving 1 week for write-up.

Risks:

- Phase D requires substantial new code (Feature Collision implementation + pretrained AlexNet-style CIFAR classifier). Will start coding in week 2 in parallel with Phase A/B.

- Statistical power: paper used n=60 victims/cell. Our budget supports paper-faithful n=60 only for Fig 4 grid; Fig 5 robustness will be n=30 (paper-faithful) and Fig 7 self-concealment may end up at n=20 (paper-faithful). Multi-class is the riskiest and may stay at n=30.

Appendix — Reproducibility artifacts

All in `final project/` repo:

- `official-metapoison/` — patched fork (TF 2.15 compat, fixed ResNet bug, re-enabled VGG)
- `src/metapoison_hpc/` — PyTorch victim trainer + CIFAR ResNet-20 implementation
- `experiments/manifest_phase_a*.csv` — 30 + 270 cell experiment grid
- `hpc_run.sbatch`, `hpc_array_craft*.sbatch`, `hpc_array_victim.sbatch` — SLURM templates
- `hpc-results/round1-job20240192/` — Round 1 (failed) full logs and metrics
- `hpc-results/round2-job20241084/` — Round 2 (success) full logs, metrics, and exported poisoned dataset
- `hpc-results/phase-a-victims/` — 24 Phase A victim experiments
- `report/build_figures.py` — figure aggregator (re-runnable as more victims complete)